

Preventing Premature Commitment in Coding Agents with an Evidence-Conditioned Execution Layer

Yisen Xu^{1,2*}, Chenglin Li^{1*}, Zehao Wang¹, Jinqiu Yang², Tse-Hsun (Peter) Chen^{1†}

¹SPEAR Lab, Concordia University, Montreal, Quebec, Canada

²O-RISA Lab, Concordia University, Montreal, Quebec, Canada

yisen.xu@mail.concordia.ca, chenglin.li@mail.concordia.ca, w_zeha@encs.concordia.ca,

jinqiu.yang@concordia.ca, peterc@encs.concordia.ca

Abstract

LLM-based coding agents often edit source code or submit patches before examining enough repository evidence to justify the change, a failure pattern we call *premature commitment*. We present *ECLoop*, an execution layer that interposes between the agent and the repository to enforce evidence-conditioned execution. For each task, *ECLoop* uses the issue description and repository structure to compile a set of conditions specifying what the agent should observe before each type of code modification or patch submission. During execution, *ECLoop* tracks which conditions the agent’s runtime trajectory has satisfied and postpones any proposed action whose required conditions remain unmet. Evaluated on all 500 instances of SWE-bench Verified with two language models and two agent scaffolds, *ECLoop* raises Pass@1 by 4.8–11.8 percentage points without model retraining or scaffold changes. Ablation experiments show that each of *ECLoop*’s three operations contributes distinct value and that structured evidence conditions outperform an equivalent natural-language summary. These gains come at no additional inference cost: by redirecting the agent before it pursues unsupported actions, *ECLoop* lowers average token consumption by up to 12.1%.

1 Introduction

Coding agents may edit code or submit a patch before examining enough repository evidence to support that action. An edit may appear plausible after inspecting the target function, even though an unseen caller, related implementation, test, or behavioral constraint would change that judgment. Likewise, a patch may appear ready for submission after one test passes while other relevant behavior remains unexamined. Thus, beyond generating a plausible repair (Zhang et al. 2024; Xia et al. 2025), an agent must determine whether it has observed enough evidence for the proposed action to proceed. We call executing an edit or submission before the relevant evidence has been examined a *premature commitment*.

Existing coding agents provide no mechanism to detect or prevent premature commitment. At each step, the agent decides what action to take based on the evidence gathered so far, but it cannot assess whether that evidence is sufficient to justify a commitment action such as editing a file or submitting a patch (Yao et al. 2023; Yang et al. 2024; OpenAI 2026a;

Wang et al. 2025b; Bouzenia, Devanbu, and Pradel 2025). Prompt-level instructions can encourage the agent to investigate more broadly before editing, but they cannot verify that the relevant evidence has actually been gathered or prevent an unsupported action from executing (Gloaguen et al. 2026; Kamath et al. 2025; Wang, Poskitt, and Sun 2025). As a result, whether a commitment action proceeds depends entirely on the agent’s own judgment, with no independent check against the evidence collected so far.

We present *ECLoop*, an execution layer that interposes between the agent’s proposed actions and their execution (Figure 1). Before each commitment action, *ECLoop* checks whether the agent has gathered sufficient evidence to justify the proposed step. If relevant evidence is missing, the action is held, and the unsatisfied conditions are returned to the agent to guide further investigation. *ECLoop* controls only whether a proposed commitment is sufficiently supported to proceed. The agent remains responsible for how it investigates the repository and constructs the repair. As an execution layer added to an existing agent, *ECLoop* requires neither model retraining nor changes to the agent’s action-selection process.

Given an issue and repository, *ECLoop* first uses a language model to compile an evidence specification from the issue description and repository structure. The specification contains conditions describing the observable evidence that should be collected before particular commitment actions proceed, such as inspecting relevant callers before editing a function or checking the available tests before submitting a patch. As the agent investigates the repository, *ECLoop* parses each command and tool output in the runtime agent trajectory into structured events, and applies deterministic satisfaction checks to check which conditions these events satisfy, without invoking a language model. *ECLoop* maintains a *global evidence gap*, consisting of all currently unsatisfied conditions, and adds this gap to the agent’s context to guide further investigation.

When the agent proposes a commitment action, *ECLoop* derives an *action-specific evidence gap* containing only the unsatisfied conditions applicable to that action. The action is postponed if this gap is nonempty and allowed to proceed otherwise. The global gap therefore guides investigation, while the action-specific gap controls execution. *ECLoop* guarantees that a commitment proceeds only after the required ob-

*These authors contributed equally.

†Corresponding author.

servable evidence has been recorded, although the agent may still misinterpret that evidence.

We evaluate *ECLoop* on all 500 instances of SWE-bench Verified (Jimenez et al. 2024) across two language models, **MiniMax-M2.5** (Minimax 2026) and **GPT-5-mini** (OpenAI 2026b), and two agent scaffolds, **mini-swe-agent v2** (Yang et al. 2024) and OpenAI’s **Codex CLI** (OpenAI 2026a). Across all four configurations, *ECLoop* improves Pass@1 by 4.8–11.8 percentage points. With mini-swe-agent v2, it raises Pass@1 from 75.8% to 80.6% under MiniMax-M2.5 and from 56.2% to 68.0% under GPT-5-mini. Integrating *ECLoop* with Codex CLI produces comparable gains, improving MiniMax-M2.5 from 74.8% to 79.8% and GPT-5-mini from 40.4% to 50.8%.

Notably, these accuracy improvements do not come at higher inference cost: by holding unsupported actions before they trigger unproductive trajectories, *ECLoop* reduces average token usage by up to 12.1%.

In summary, this work makes the following contributions:

- We identify *premature commitment* as a distinct failure mode in coding agents and formulate evidence-conditioned execution, which separates proposing an action from deciding whether sufficient evidence has been collected for it to proceed.
- We introduce *ECLoop*, an execution control layer that tracks task-specific evidence, guides the agent using what remains missing, and checks relevant evidence before edits and final submissions.
- On all 500 SWE-bench Verified instances, *ECLoop* improves Pass@1 for both GPT-5-mini and MiniMax-M2.5 across mini-swe-agent v2 and Codex CLI, with gains of 4.8–11.8 percentage points and a best Pass@1 of 80.6%. Across all four model–scaffold configurations, *ECLoop* also reduces token usage by 1.4–12.1% and inference cost by 1.5–10.2%.

2 Related Work

2.1 Evidence Gathering in Coding Agents

Recent studies show that coding agents frequently produce patches without sufficient repository evidence. Agents generate edits for already-fixed issues (Gloaguen et al. 2026), miss relevant repository context that would change the repair (Zhang et al. 2026b), and produce incomplete or test-overfitted patches due to insufficient issue and code understanding (Pabba et al. 2025). These findings show that repair quality depends not only on patch generation but also on what evidence the agent gathers before committing to a change.

Several systems address this by building evidence checks into multi-stage repair pipelines. *EVIACT* (Meng et al. 2026) coordinates a retrieval scaffold for localization, a compile gate for rejecting invalid patches, and a test-driven gate for validating target-test recovery across a staged pipeline. *SWE-DOCTOR* (Guo et al. 2026b) requires a reproduction test to pass before patch generation, following work that treats reproduction tests as executable specifications (Mündler et al. 2024; Ahmed et al. 2025, 2026; Zhang et al. 2026a). In these systems, the required evidence is tied to specific tools and

artifacts, such as compiler outcomes, reproduction tests, or target-test results, and is checked at predefined points in the workflow.

ECLoop instead derives evidence conditions from each issue, so different tasks require different evidence rather than passing through the same fixed gates. It evaluates these conditions at every commitment boundary and operates as an execution layer on top of an existing scaffold, without modifying the scaffold’s action-selection policy.

2.2 Runtime Enforcement for LLM Agents

Runtime enforcement systems constrain agent behavior during execution. *AGENTSPEC* enforces specified constraints on individual actions, while *AGENT-C* enforces temporal constraints over sequences of tool calls (Wang, Poskitt, and Sun 2025; Kamath et al. 2025). *PROBGUARD* predicts trajectories that may reach unsafe states (Wang et al. 2026). Other systems generate, adapt, or verify guard policies (Miculicich et al. 2025; Luo et al. 2025; Xiang et al. 2025), while *MI9* and *NEMO GUARDRAILS* apply controls across multiple system layers (Wang et al. 2025a; Rebedea et al. 2023). *TRIAD* extends proceed-or-refuse enforcement with an update decision and verbal feedback that helps the agent revise unsafe plans (Sun et al. 2026b).

Despite their different implementations, these systems primarily enforce *behavioral compliance*: whether an action or trajectory satisfies a safety, security, or governance policy. However, an edit may violate no such policy and follow every required tool order while still being premature because the agent has not established that the change is needed.

ECLoop addresses this distinct failure mode by enforcing *evidence-conditioned execution*. It checks whether the agent’s trajectory contains the observations needed to justify the current repair task, rather than whether the action satisfies a fixed policy. Although *TRIAD* also provides verbal feedback, its feedback remediates unsafe plans, whereas *ECLoop*’s feedback directs the agent toward the specific repository investigation still needed.

2.3 Intervening on Agent Trajectories

Several methods improve agent trajectories by helping agents choose better actions. *REACT* (Yao et al. 2023) interleaves reasoning traces with actions, allowing the agent to update its plan based on new observations. *REFLEXION* (Shinn et al. 2023) uses verbal self-reflection to improve later trials. Process reward models score intermediate steps to guide better action choices (Xi et al. 2026; Han et al. 2026). These methods primarily improve which action the agent proposes, yet agents may recognize that a tool is needed and still fail to call it during generation (Sun et al. 2026a).

Premature commitment is related to the classical least-commitment principle, which delays decisions until available constraints require them (Weld 1994). Direct intervention also carries risk: it may recover failing trajectories but disrupt ones that would otherwise succeed (Vasudev et al. 2026).

ECLoop instead addresses a different question: whether the agent is ready to commit. It does not choose the agent’s next action, but checks whether the agent has gathered the

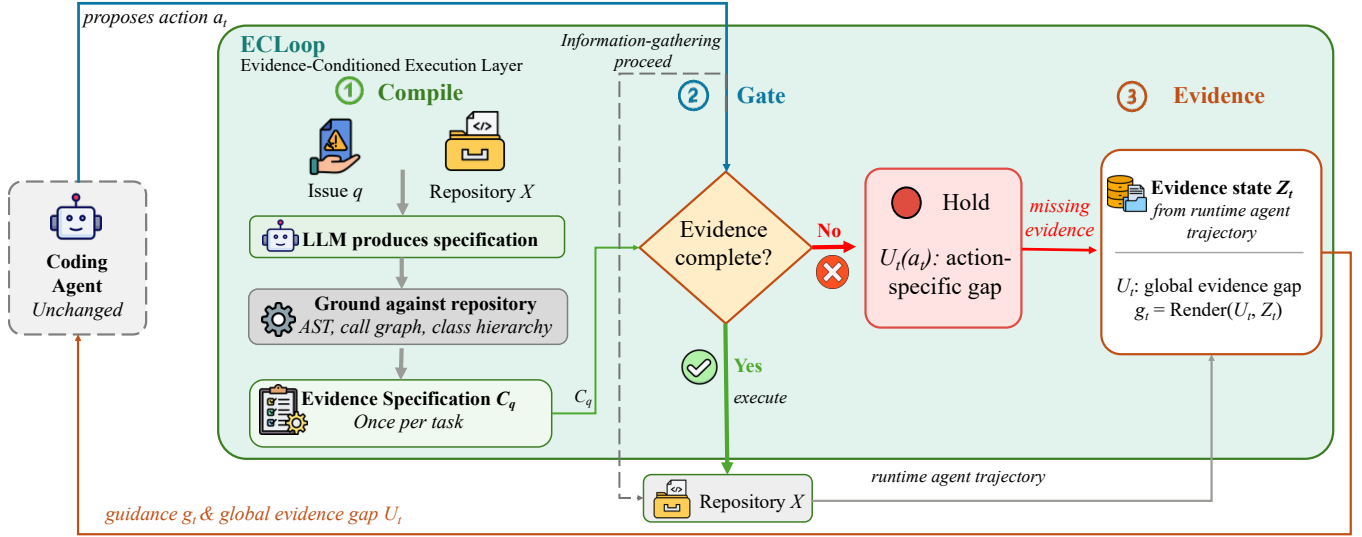


Figure 1: **ECLoop** overview. *ECLoop* interposes between the coding agent (left) and the repository: (1) compiling evidence conditions from the issue once per task, (2) gating commitment actions whose action-specific evidence gap remains nonempty, and (3) returning the global evidence gap as guidance until all conditions are met. Information-gathering actions proceed freely.

task-specific evidence needed to justify an edit or final submission.

3 *ECLoop*: Evidence-Conditioned Execution

A coding agent may propose a plausible action before completing the investigation needed to support it (i.e., the *evidence*). For example, it may identify a likely edit while relevant callers, related implementations, or failure conditions remain unexamined. Conventional agent runtimes typically execute a proposed action without separately assessing whether the available evidence is sufficient. This conflates two decisions: whether the action is a plausible next step and whether the agent is ready to commit to it.

We introduce *ECLoop*, an evidence-conditioned execution layer that separates action proposal from execution readiness. The coding agent continues to decide what to do next, while *ECLoop* tracks task-specific evidence from the runtime agent trajectory and determines whether actions that modify or finalize the solution should proceed. **Information-gathering actions**, including file reads, code searches, and diagnostic commands, proceed without restriction. In contrast, **commitment actions**, such as source-code edits and final submission, proceed only when supported by the relevant evidence.

ECLoop maintains two views of the evidence still missing from the ongoing investigation, which we refer to as the *evidence gap*. The *global evidence gap* is added to the model context so the agent can see what evidence the task still needs. When the agent proposes a commitment action, *ECLoop* derives an *action-specific evidence gap* containing only the unsatisfied conditions relevant to that action, and gates the action based on whether those conditions have been met. Beyond these two interventions, the underlying agent remains unchanged.

As illustrated in Figure 1, *ECLoop* operates in three stages.

It first analyzes the issue and repository to compile task-specific conditions describing the evidence needed before a commitment action can proceed. During execution, it uses the runtime agent trajectory to determine which conditions have been satisfied and adds the remaining unsatisfied evidence gap to the model context. When the agent proposes an edit or final submission, *ECLoop* checks only the conditions relevant to that action. If any remain unsatisfied, it holds the action and returns unsatisfied conditions to guide the agent’s next action.

3.1 Compiling the Evidence Specification

Given an issue q and repository X , *ECLoop* constructs a task-specific evidence specification that defines what the agent must establish before a commitment action can proceed. This compilation separates two questions: which evidence is needed for the task, and how its satisfaction can be determined from the repository and the runtime agent trajectory. We present *ECLoop* in the context of software repair; applying it to other domains would require domain-specific representations of evidence and corresponding procedures for evaluating them.

To identify the evidence relevant to the task, *ECLoop* uses a language model to analyze the issue together with structural information extracted from the repository and produce a task-specific evidence specification:

$$C_q = \{c_1, \dots, c_m\}, \quad (1)$$

where each condition c_i specifies an investigation requirement and the commitment actions to which it applies. At this stage, a condition may specify an abstract program role without yet resolving it to a concrete repository entity. For example, “the function to be edited,” “its callers,” and “related implementations” describe what must be inspected, but not which functions or files satisfy those roles in the current task.

The specification therefore defines what evidence is needed without assuming that the relevant entities are already known.

Formal evidence representation. Free-form instructions are inefficient for execution control because they do not explicitly specify how required evidence should be tracked, when it applies, or how its completion should be determined. *ECLoop* therefore represents each evidence condition as a structured specification (Wang, Poskitt, and Sun 2025; Guo et al. 2026a), explicitly recording the relevant program entity, the observable evidence required, the commitment action to which it applies, and a satisfaction predicate.

ECLoop converts each abstract condition in C_q into a repository-specific condition by identifying the concrete program entities to which it applies. To perform this resolution, *ECLoop* uses abstract syntax tree traversal to locate relevant functions and classes, the call graph to identify caller–callee relationships, and the class hierarchy to identify related or overriding implementations. Each condition c_i is then represented as

$$\phi_i = \langle b_i, v_i, R_i, \text{sat}_i \rangle, \quad (2)$$

where b_i specifies the type of commitment action at which the condition is checked, v_i denotes the concrete program entity to which it applies, R_i specifies the event pattern that must appear in the runtime agent trajectory, and $\text{sat}_i(Z_t) \in \{0, 1\}$ indicates whether the event pattern R_i has been matched in the current evidence state Z_t .

This representation allows each condition to be evaluated against the runtime agent trajectory, presented to the agent as guidance, and checked when a commitment action is proposed. By preserving individual conditions throughout execution, *ECLoop* avoids the ambiguity introduced by compressing the remaining evidence into an unconstrained natural-language summary.

Satisfaction is computed from the runtime agent trajectory, such as inspected code locations, executed commands, and diagnostic outcomes, rather than from the model claiming that it completed an investigation. Grounding succeeds only when the references in a condition can be resolved and its satisfaction can be checked:

$$\text{Ground}(c_i, X) = \begin{cases} \phi_i, & \text{if } c_i \text{ resolves to checkable evidence,} \\ \perp, & \text{otherwise.} \end{cases} \quad (3)$$

Conditions that cannot be resolved to concrete repository entities are removed from the specification. The language model therefore proposes candidate conditions, but does not determine their execution-time enforcement. A condition becomes active only after its references have been resolved, and its satisfaction is evaluated deterministically from the runtime agent trajectory. Errors in the generated specification may reduce coverage, but they cannot introduce checks that the system cannot evaluate.

Some conditions can be resolved before execution because the issue already identifies the relevant program entity or failure. Others can be resolved only after the agent proposes a specific action. For example, a condition requiring caller inspection cannot identify the relevant callers until the agent proposes editing a function f . *ECLoop* then uses f to retrieve

its callers from the call graph and checks the runtime agent trajectory to determine whether they have been inspected.

Let C_t denote the grounded conditions available by step t . Before evaluating a proposed commitment action, *ECLoop* grounds any additional conditions whose references can now be resolved from that action and adds them to C_t . Thus, C_q captures the task-specific investigation requirements, while C_t contains the concrete conditions that can be evaluated during execution.

3.2 Maintaining the Global Evidence Gap

At step t , *ECLoop* derives the current evidence state from the runtime agent trajectory:

$$Z_t = \text{Observe}(\tau_t). \quad (4)$$

ECLoop represents each executed command and its output as observable events describing what the agent inspected, searched, modified, or executed. These events record the relevant code locations, symbols, files, line ranges, and execution outcomes. Each predicate sat_i specifies the event pattern required to satisfy its condition. For example, an inspection condition requires an event showing that the relevant code location was viewed, while a reproduction condition requires a recognized failure outcome before the edit. Hence, satisfaction is determined directly from the recorded trajectory using fixed, condition-specific criteria.

ECLoop evaluates the grounded conditions in C_t against Z_t and collects those that remain unsatisfied:

$$U_t = \{\phi_i \in C_t \mid \text{sat}_i(Z_t) = 0\}. \quad (5)$$

We refer to U_t as the **global evidence gap**. It summarizes the task-relevant evidence that remains missing at this point in the trajectory.

ECLoop converts the remaining evidence gap into guidance for the agent:

$$g_t = \text{Render}(U_t, Z_t), \quad (6)$$

where g_t describes what still needs to be established and refers to concrete program entities when available. Conditions that have already been satisfied are omitted, so the agent receives only the evidence that remains missing rather than the full initial specification.

The model generates its next response conditioned on the issue, the runtime agent trajectory, and the current evidence gap:

$$y_{t,k} \sim p_\theta(\cdot \mid q, \tau_t, g_t, y_{t,<k}). \quad (7)$$

Here, τ_t provides the execution history, while g_t explicitly identifies the evidence that remains missing. Conditioning generation on g_t directs the agent toward unresolved investigation requirements without prescribing a specific action or order. The agent therefore retains control over how to obtain the required evidence.

After an executed action a_t returns observation o_t , *ECLoop* updates the trajectory and evidence state:

$$\tau_{t+1} = \tau_t \oplus \langle a_t, o_t \rangle, \quad Z_{t+1} = \text{Observe}(\tau_{t+1}). \quad (8)$$

The updated state produces a new global evidence gap for the next step. Evidence is therefore maintained as a dynamic execution state rather than as a static instruction supplied only at the beginning of the task.

3.3 Gating Commitment by the Action-Specific Gap

The global evidence gap guides the agent toward what remains to be established, but guidance alone cannot prevent a premature commitment. When the model proposes a commitment action, *ECLoop* first uses that action to resolve any condition references that could not be identified earlier and adds the newly resolved conditions to C_t . It then checks whether the evidence required for that action is complete.

For a proposed action a_t , *ECLoop* derives the action-specific evidence gap:

$$U_t(a_t) = \{\phi_i \in C_t \mid \text{applies}(\phi_i, a_t) \wedge \text{sat}_i(Z_t) = 0\}, \quad (9)$$

where $\text{applies}(\phi_i, a_t)$ holds when a_t matches the action type b_i and affects the entity v_i . Unlike the global gap U_t , which guides the overall investigation, $U_t(a_t)$ contains only the missing evidence relevant to the proposed action.

A commitment action may proceed only when its action-specific evidence gap is empty:

$$U_t(a_t) = \emptyset. \quad (10)$$

Non-commitment actions are unaffected by this check. When a commitment action has unmet evidence requirements, *ECLoop* postpones it and adds the missing evidence to the next model context. The agent may then continue the investigation, revise the proposed action, or pursue a different approach. Thus, *ECLoop* constrains when commitment is allowed without prescribing the subsequent action.

Execution control and guarantees. The global evidence gap guides the agent’s investigation, new observations update the evidence state, and the action-specific gap determines whether a proposed commitment may proceed. As an execution control layer, *ECLoop* determines whether a proposed commitment action is ready to proceed without deciding which action the agent should propose.

ECLoop enforces evidence completion within a bounded hold budget. When a commitment action’s applicable conditions remain unsatisfied, *ECLoop* postpones the action and returns the missing evidence to the agent, repeating this up to three times for the same commitment target. If the budget is exhausted without the conditions being met, the action proceeds through an audited fallback in which any unsatisfied conditions remain recorded, preserving the distinction between an evidence-supported commitment and one allowed by the fallback.

4 Experiments

We evaluate whether *ECLoop* improves repository-level issue resolution across large language models and agent scaffolds, how much execution overhead and token cost it adds, and how much each component contributes.

Benchmark and metric. We evaluate *ECLoop* on all 500 instances of **SWE-bench Verified** (Jimenez et al. 2024), a human-validated subset of SWE-bench in which each instance pairs a real GitHub issue with a repository snapshot and held-out tests. We report **Pass@1**, the fraction of instances resolved by a single agent run. Because stronger

Model	Method	Pass@1	Δ	RRU
GPT-5-mini	Baseline	56.2%	—	—
	Self-Refine	54.8%	−1.4	−3.2%
	<i>ECLoop</i>	68.0%	+11.8	26.9%
MiniMax-M2.5 [†]	Baseline	75.8%	—	—
	Self-Refine	74.0%	−1.8	−7.4%
	<i>ECLoop</i>	80.6%	+4.8	19.8%

Table 1: Main results on SWE-bench Verified (500 instances) with mini-swe-agent v2. Both *ECLoop* improvements are significant under an exact McNemar test (McNemar 1947) ($p < 0.001$).

[†]High reasoning effort. RRU = relative reduction in unresolved instances. Self-Refine (Madaan et al. 2023) uses up to three revision iterations.

models/agents leave fewer failures available to recover, we also report the **relative reduction in unresolved instances** (RRU) (Manning and Schütze 1999),

$$\text{RRU} = \frac{\text{Pass@1}_{\text{ECLoop}} - \text{Pass@1}_{\text{base}}}{1 - \text{Pass@1}_{\text{base}}}.$$

For each comparison, statistical significance is assessed using an exact McNemar test (McNemar 1947) on the 500 paired binary outcomes.

Models. We use **MiniMax-M2.5** with high reasoning effort (Minimax 2026) and **GPT-5-mini** (OpenAI 2026b). Both offer strong coding performance at a cost that enables full-benchmark evaluation. We apply the same model configuration across all experiments.

Agent scaffolds. Both models run on *mini-swe-agent v2* (Yang et al. 2024), a lightweight scaffold released by the SWE-agent project (Yang et al. 2024). To test whether *ECLoop* also improves a full-featured production coding agent in addition to a minimal research scaffold, we additionally integrate *ECLoop* with **OpenAI Codex CLI** v0.144.4 (OpenAI 2026a) and run both models.

Baselines and controlled comparison. *ECLoop* acts as an execution layer on top of the unchanged agent. Within each configuration, the baseline and *ECLoop* share the same model, prompt, tools, and scaffold. The only addition is that *ECLoop* compiles task-specific evidence conditions with the same model, exposes the global evidence gap U_t , and checks the action-specific evidence gap $U_t(a_t)$ before each commitment action proceeds.

In addition to the baseline agents, we compare *ECLoop* against Self-Refine (Madaan et al. 2023), an iterative refinement method in which the model reviews the agent’s own patch and revises it up to three iterations. Self-Refine uses comparable additional inference but operates after the agent has already produced a patch, rather than gating commitment actions during the agent loop.

4.1 *ECLoop* Improves Repair Across LLMs

ECLoop improves both models under mini-swe-agent v2 (Table 1). For GPT-5-mini, Pass@1 increases from 56.2% to

Model	Base	<i>ECLoop</i>	Δ	RRU
GPT-5-mini	40.4%	50.8%	+10.4	17.4%
MiniMax-M2.5 [†]	74.8%	79.8%	+5.0	19.8%

Table 2: Results of integrating *ECLoop* into Codex CLI on SWE-bench Verified (500 instances). Both improvements are statistically significant (two-sided exact McNemar test (McNemar 1947), $p < 0.001$).

[†]High reasoning effort. RRU = relative reduction in unresolved instances.

68.0% (+11.8 percentage points, pp), corresponding to an RRU of 26.9%. For MiniMax-M2.5, Pass@1 increases from 75.8% to 80.6% (+4.8 pp), corresponding to an RRU of 19.8%. Despite their different baseline performance, the two models achieve RRUs of 26.9% and 19.8%, showing that *ECLoop* provides substantial relative gains for lower- and higher-performing baselines.

Compared with the baseline agents, *ECLoop* newly resolves 33 and 68 failures for MiniMax-M2.5 and GPT-5-mini, respectively, while regressing on only 9 successes in each case. The few regressions occur when the agent repeatedly proposes a commitment action that the gate holds for insufficient evidence, exhausting the hold limit of three; the fallback release then permits an under-supported action that produces an incorrect patch.

Integrating Self-Refine with the mini-swe-agent v2 slightly degrades Pass@1 in both models (by -1.4 and -1.8 pp), confirming that post hoc self-review cannot recover from decisions made on insufficient evidence. *ECLoop* avoids this by gating commitment actions before they execute, preventing premature commitment rather than attempting to correct it afterward.

4.2 Integrating *ECLoop* into Codex CLI

To evaluate whether *ECLoop* transfers beyond mini-swe-agent v2, we integrate it into Codex CLI (OpenAI 2026a) through its hook mechanism. The hook intercepts proposed commitment actions before execution, checks them against the accumulated evidence, and either permits the action or returns feedback identifying the missing observation. This integration requires no modification to Codex CLI’s model, tool set, or action-selection policy.

ECLoop improves both models under Codex CLI (Table 2). For GPT-5-mini, Pass@1 increases by 10.4 pp, corresponding to an RRU of 17.4%; *ECLoop* newly resolves 68 baseline failures while regressing on 16 successes. For MiniMax-M2.5, Pass@1 increases by 5.0 pp, corresponding to an RRU of 19.8%; *ECLoop* newly resolves 35 failures while regressing on 10 successes. These Pass@1 gains closely match those under mini-swe-agent v2 (+11.8 and +4.8 pp), showing that *ECLoop* provides similar improvements across different coding-agent scaffolds.

4.3 Execution Efficiency and Cost

Table 3 reports per-instance token usage and monetary cost averaged over all 500 instances. *ECLoop* reduces both token

usages and cost in all four configurations, with cost savings of 1.5–10.2% and token reductions of 1.4–12.1%. The reported totals include the per-task specification-compilation call. In short, *ECLoop* achieves the Pass@1 gains without increasing inference cost. Although *ECLoop* introduces additional evidence checks, it reduces overall token use by redirecting the agent before unsupported commitment actions lead to longer, less productive trajectories.

The largest savings occur in the weak baseline configuration. With mini-swe-agent v2 and GPT-5-mini, which has the baseline Pass@1 at 56.2%, *ECLoop* reduces token use by 12.1% and cost by 10.2%. In contrast, with Codex CLI and MiniMax-M2.5, which have the higher baseline Pass@1 at 74.8%, the reductions are 1.4% and 1.5%, respectively. This pattern suggests that *ECLoop* may provide better efficiency gains when the baseline is more likely to spend computational resources on unsuccessful trajectories.

4.4 Ablation Analysis

ECLoop has three operations: *guidance*, which communicates the remaining evidence gap to the agent; the *commitment check*, which blocks a commitment action while its relevant evidence remains incomplete; and the *evidence-state update*, which recomputes the evidence state after each observation. We ablate each on a fixed, randomly sampled subset of 100 instances with GPT-5-mini.

As shown in Table 4, removing any single operation from full *ECLoop* (68 solved) degrades performance. The commitment check contributes the most (-10 pp to 58%), while the evidence-state update is nearly as important (-9 pp to 59%). Without evidence-state update, newly observed evidence is not reflected in the gaps, weakening both guidance and checking. Guidance adds a further 5 pp. All three together outperform every partial variant, confirming that they provide complementary benefits. Replacing the structured specification with a natural-language summary also drops the Pass@1 by 10 pp, to 58%, falling below even no guidance (Pass@1 of 63%).

Limitations

Dependence on issue quality. The evidence specification C_q is compiled from the issue description and repository structure, so its completeness is bounded by how well the issue characterizes the underlying problem. When the issue is vague or under-specified, C_q may omit conditions that matter for a correct fix or include irrelevant ones, weakening the gate’s signal. The issues in SWE-bench Verified are human-validated and reasonably well-specified, which bounds this effect in our experiments. On noisier real-world trackers, a preceding issue-clarification step could improve C_q quality.

Shared-model evidence operations. The evidence specification, gap assessment, and guidance are all produced by the same model that drives the agent. A blind spot the model has at action-selection time it may also have when assessing evidence sufficiency. We partially mitigate this by grounding C_q against repository structure rather than relying on the model’s parametric knowledge alone, but using a stronger

Model	Avg. Tokens (K)		Avg. Cost		Total Cost	
	Base	<i>ECLoop</i>	Base	<i>ECLoop</i>	Base	<i>ECLoop</i>
<i>mini-swe-agent v2</i>						
GPT-5-mini	179	157–12.1%	\$0.047	\$0.042–10.2%	\$23.60	\$21.18
MiniMax-M2.5 [†]	219	206–6.2%	\$0.073	\$0.069–6.1%	\$36.64	\$34.39
<i>Codex CLI</i>						
GPT-5-mini	111	105–5.4%	\$0.030	\$0.029–5.3%	\$15.16	\$14.36
MiniMax-M2.5 [†]	165	162–1.4%	\$0.055	\$0.054–1.5%	\$27.54	\$27.14

Table 3: Execution efficiency on SWE-bench Verified (per-instance averages over 500 instances; costs in USD).

[†]High reasoning effort.

Configuration	Solved	Δ
Full <i>ECLoop</i>	68	–
– guidance	63	–5
– commitment check	58	–10
– state update	59	–9
Baseline (– all)	47	–21
spec $\rightarrow$ natural language	58	–10

Table 4: Ablation on a fixed 100-instance subset using mini-swe-agent v2 and GPT-5-mini. Each row removes or replaces one component of full *ECLoop*.

or independent model for evidence operations is a natural extension that *ECLoop*’s design already accommodates.

Evaluation scope. We evaluate on SWE-bench Verified with two models and two scaffolds. While this establishes cross-model and cross-scaffold generality, it does not cover other languages, task types such as feature addition, or benchmarks beyond SWE-bench.

5 Conclusion

We presented *ECLoop*, an evidence-conditioned execution layer that addresses premature commitment in coding agents by tracking task-specific evidence and gating commitment actions whose supporting conditions remain unsatisfied. On SWE-bench Verified, *ECLoop* improves Pass@1 by 4.8–11.8 percentage points across two models and two scaffolds while reducing token usage by up to 12.1%, without modifying the underlying agent. The results demonstrate that controlling *when* an agent may commit is an effective and lightweight complement to improving *what* it chooses to do.

References

Ahmed, T.; Ganhotra, J.; Pan, R.; Shinnar, A.; Sinha, S.; and Hirzel, M. 2025. Otter: Generating Tests from Issues to Validate SWE Patches. In Singh, A.; Fazel, M.; Hsu, D.; Lacoste-Julien, S.; Berkenkamp, F.; Maharaj, T.; Wagstaff, K.; and Zhu, J., eds., *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada,*

July 13-19, 2025, volume 267 of *Proceedings of Machine Learning Research*. PMLR / OpenReview.net.

Ahmed, T.; Ganhotra, J.; Shinnar, A.; and Hirzel, M. 2026. Reproduction Test Generation for Java SWE Issues. *CoRR*, abs/2605.04320.

Bouzenia, I.; Devanbu, P. T.; and Pradel, M. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*, 2188–2200. IEEE.

Gloaguen, T.; Mündler, N.; Müller, M. N.; Raychev, V.; and Vechev, M. T. 2026. Coding Agents Don’t Know When to Act. *CoRR*, abs/2605.07769.

Guo, L.; Liu, W.; Heng, Y. W.; Chen, T. P.; and Wang, Y. 2026a. Agent-SAMA: State-Aware Mobile Assistant. In Koenig, S.; Jenkins, C.; and Taylor, M. E., eds., *Fortieth AAAI Conference on Artificial Intelligence, Thirty-Eighth Conference on Innovative Applications of Artificial Intelligence, Sixteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2026, Singapore, January 20-27, 2026*, 29459–29467. AAAI Press.

Guo, Y.; Liu, Y.; Zhang, J. M.; Ma, Y.; Lou, Y.; and Chen, Z. 2026b. SWE-Doctor: Guiding Software Engineering Agents with Runtime Diagnosis from Multi-Faceted Bug Reproduction Tests. arXiv:2607.00990.

Han, H.; Xie, J.; Ma, X.; Zhu, W.; Zhang, Z.; Long, Z.; Chen, H.; and Ye, Q. 2026. SWE-TRACE: Optimizing Long-Horizon SWE Agents Through Rubric Process Reward Models and Heuristic Test-Time Scaling. *CoRR*, abs/2604.14820.

Jimenez, C. E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; and Narasimhan, K. R. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues? In *The Twelfth International Conference on Learning Representations*.

Kamath, A.; Zhang, S.; Xu, C.; Ugare, S.; Singh, G.; and Misailovic, S. 2025. Enforcing Temporal Constraints for LLM Agents. *CoRR*, abs/2512.23738.

Luo, W.; Dai, S.; Liu, X.; Banerjee, S.; Sun, H.; Chen, M.; and Xiao, C. 2025. AGrail: A Lifelong Agent Guardrail with Effective and Adaptive Safety Detection. In Che, W.; Nabende, J.; Shutova, E.; and Pilehvar, M. T., eds., *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL*

- 2025, Vienna, Austria, July 27 - August 1, 2025, 8104–8139. Association for Computational Linguistics.
- Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhunoy, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In Oh, A.; Naumann, T.; Globerson, A.; Saenko, K.; Hardt, M.; and Levine, S., eds., *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
- Manning, C. D.; and Schütze, H. 1999. *Foundations of statistical natural language processing*. MIT Press. ISBN 978-0-262-13360-9.
- McNemar, Q. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika*, 12(2): 153–157.
- Meng, Q.; Zhang, X.; Ren, Z.; and Visser, J. 2026. EviACT: An Evidence-to-Action Framework for Agentic Program Repair. *CoRR*, abs/2605.27238.
- Miculicich, L.; Parmar, M.; Palangi, H.; Dvijotham, K. D.; Montanari, M.; Pfister, T.; and Le, L. T. 2025. VeriGuard: Enhancing LLM Agent Safety via Verified Code Generation. *CoRR*, abs/2510.05156.
- Minimax. 2026. MiniMax M2.5. <https://www.minimax.io/news/minimax-m25/>.
- Mündler, N.; Müller, M. N.; He, J.; and Vechev, M. T. 2024. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. In Globerson, A.; Mackey, L.; Belgrave, D.; Fan, A.; Paquet, U.; Tomczak, J. M.; and Zhang, C., eds., *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.
- OpenAI. 2026a. Codex CLI. <https://github.com/openai/codex>.
- OpenAI. 2026b. OpenAI GPT-5 System Card. *CoRR*, abs/2601.03267.
- Pabba, A.; Chen, S.; Mathai, A.; Chakraborty, A.; and Ray, B. 2025. REFINE: Enhancing Program Repair Agents through Context-Aware Patch Refinement. *CoRR*, abs/2510.03588.
- Reborea, T.; Dinu, R.; Sreedhar, M. N.; Parisien, C.; and Cohen, J. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. In Feng, Y.; and Lefever, E., eds., *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023 - System Demonstrations, Singapore, December 6-10, 2023*, 431–445. Association for Computational Linguistics.
- Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: language agents with verbal reinforcement learning. In Oh, A.; Naumann, T.; Globerson, A.; Saenko, K.; Hardt, M.; and Levine, S., eds., *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
- Sun, C.; Liu, L.; Yan, G.; Wang, Z.; and Weng, T. 2026a. LLM Agents Already Know When to Call Tools - Even Without Reasoning. *CoRR*, abs/2605.09252.
- Sun, Y.; Zhang, J.; Cohn, S.; Zhang, Z.; Liu, F.; and Yuan, X. 2026b. From Risk Classification to Action Plan Remediation: A Guardrail Feedback Driven Framework for LLM Agents. *CoRR*, abs/2606.05805.
- Vasudev, R.; Russak, M.; Bikel, D.; and AlShikh, W. 2026. Accurate Failure Prediction in Agents Does Not Imply Effective Failure Prevention. *CoRR*, abs/2602.03338.
- Wang, C. L.; Singhal, T.; Kelkar, A.; and Tuo, J. 2025a. MI9: An Integrated Runtime Governance Framework for Agentic AI. arXiv:2508.03858.
- Wang, H.; Poskitt, C. M.; and Sun, J. 2025. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. *CoRR*, abs/2503.18666.
- Wang, H.; Poskitt, C. M.; Wei, J.; and Sun, J. 2026. ProGuard: Probabilistic Runtime Monitoring for LLM Agent Safety. arXiv:2508.00500.
- Wang, X.; Li, B.; Song, Y.; Xu, F. F.; Tang, X.; Zhuge, M.; Pan, J.; Song, Y.; Li, B.; Singh, J.; Tran, H. H.; Li, F.; Ma, R.; Zheng, M.; Qian, B.; Shao, Y.; Muennighoff, N.; Zhang, Y.; Hui, B.; and Lin, J. 2025b. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net.
- Weld, D. S. 1994. An Introduction to Least Commitment Planning. *AI Mag.*, 15(4): 27–61.
- Xi, Z.; Liao, C.; Li, G.; Zhang, Z.; Chen, W.; Wang, B.; Jin, S.; Zhou, Y.; Guan, J.; Wu, W.; Ji, T.; Gui, T.; Zhang, Q.; and Huang, X. 2026. AgentPRM: Process Reward Models for LLM Agents via Step-Wise Promise and Progress. In Hacı, H.; Maarek, Y.; Bonchi, F.; Guy, I.; and Yilmaz, E., eds., *Proceedings of the ACM Web Conference 2026, WWW 2026, Dubai, United Arab Emirates, originally scheduled for April 13-17, 2026, rescheduled for June 29 - July 3, 2026*, 4184–4195. ACM.
- Xia, C. S.; Deng, Y.; Dunn, S.; and Zhang, L. 2025. Demystifying LLM-Based Software Engineering Agents. *Proc. ACM Softw. Eng.*, 2(FSE): 801–824.
- Xiang, Z.; Zheng, L.; Li, Y.; Hong, J.; Li, Q.; Xie, H.; Zhang, J.; Xiong, Z.; Xie, C.; Yang, C.; Song, D.; and Li, B. 2025. GuardAgent: Safeguard LLM Agents via Knowledge-Enabled Reasoning. In Singh, A.; Fazel, M.; Hsu, D.; Lacoste-Julien, S.; Berkenkamp, F.; Maharaj, T.; Wagstaff, K.; and Zhu, J., eds., *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, volume 267 of *Proceedings of Machine Learning Research*. PMLR / OpenReview.net.
- Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K. R.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and

Acting in Language Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

Zhang, Q.; Zheng, Y.; Shang, Y.; Sun, W.; Hu, H.; Fang, C.; Chen, Z.; and Xiao, L. 2026a. ReProAgent: Tool-Augmented Multi-Stage Agentic Generation of Bug Reproduction Tests from Issue Reports. arXiv:2607.09123.

Zhang, S.; Wang, Y.; Liang, J.; Shi, Y.; Zeng, W.; Wang, M.; He, S.; Xu, N.; Ye, S.; Cai, K.; and Gu, X. 2026b. SWE-Explore: Benchmarking How Coding Agents Explore Repositories. *CoRR*, abs/2606.07297.

Zhang, Y.; Ruan, H.; Fan, Z.; and Roychoudhury, A. 2024. AutoCodeRover: Autonomous Program Improvement. In Christakis, M.; and Pradel, M., eds., *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, 1592–1604. ACM.